Day – 18 Implementation of CNN

A Convolutional Neural Network (CNN) is a deep learning algorithm mainly used for image classification. In this implementation, CNN is used to classify handwritten digits from the MNIST dataset.

Dataset

The MNIST dataset contains grayscale images of handwritten digits from 0 to 9. Each image has a size of 28 × 28 pixels. In this implementation, 10,000 images were used for training and 2,000 images for testing.

Image Normalization

The pixel values range from 0 to 255. They were divided by 255 to convert them into the range 0 to 1. This helps the model train more efficiently.

Image Reshaping

The images were reshaped from:

28 × 28 → 28 × 28 × 1

The 1 represents the grayscale channel.

Convolutional Layer

The CNN uses two convolutional layers:

- First layer: 32 filters of size 3 × 3
- Second layer: 64 filters of size 3 × 3

These layers extract important features such as edges, curves, and shapes from the handwritten images.

ReLU Activation

The convolutional layers use the ReLU (Rectified Linear Unit) activation function.

Its formula is:

ReLU(x) = max(0, x)

It introduces non-linearity and helps the network learn complex patterns.

Max Pooling

Two 2 × 2 MaxPooling layers are used. Max pooling reduces the size of feature maps while retaining the most important information.

Flatten Layer

The Flatten layer converts the extracted feature maps into a one-dimensional vector, which can then be passed to the dense layers.

Dense Layers

A dense layer with 64 neurons is used to learn from the extracted features.

The final dense layer contains 10 neurons, representing the ten digit classes:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9

Softmax

The final layer uses the Softmax activation function. It produces probabilities for all 10 classes, and the class with the highest probability is selected as the prediction.

Model Compilation

The model was compiled using:

- Optimizer: Adam
- Loss Function: Sparse Categorical Crossentropy
- Metric: Accuracy

Model Training

The CNN was trained for 10 epochs with a batch size of 32. 20% of the training data was used for validation.

Model Evaluation

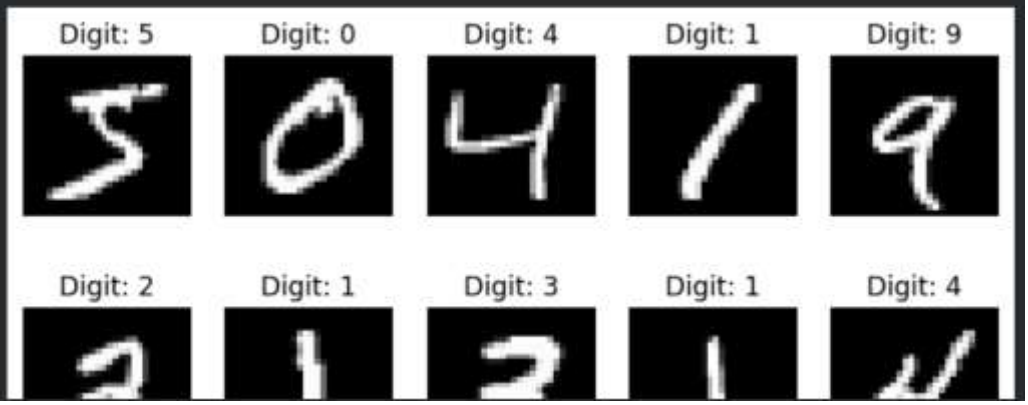
After training, the model was evaluated on the 2,000 test images. The implementation used:

- Test Accuracy
- Confusion Matrix
- Classification Report
- Actual vs Predicted Image Visualization

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
from tensorflow.keras import datasets
(X_train, y_train), (X_test, y_test) = datasets.mnist.load_data()
# Use a smaller dataset
X_train = X_train[:10000]
y_train = y_train[:10000]
X_test = X_test[:2000]
y_test = y_test[:2000]
print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (10000, 28, 28)
Testing data: (2000, 28, 28)
```

```python
plt.figure(figsize=(8, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_train[i], cmap="gray")
    plt.title(f"Digit: {y_train[i]}")
    plt.axis("off")
plt.show()
```




```python
X_train = X_train / 255.0
X_test = X_test / 255.0
```

```python
X_train = X_train.reshape(-1, 28, 28, 1)
X_test = X_test.reshape(-1, 28, 28, 1)

print("X_train:", X_train.shape)
print("X_test:", X_test.shape)
```

```
X_train: (10000, 28, 28, 1)
X_test: (2000, 28, 28, 1)
```

```python
from tensorflow.keras import models, layers
cnn_model = models.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(32, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation="relu"),
    layers.MaxPooling2D((2, 2)),
    layers.Flatten(),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
```

```python
cnn_model.summary()
```

```
Model: "sequential_2"
```

Layer (type)	Output Shape	Param #
conv2d_4 (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d_4 (MaxPooling2D)	(None, 13, 13, 32)	0

```
Total params: 121,930 (476.29 KB)
Trainable params: 121,930 (476.29 KB)
Non-trainable params: 0 (0.00 B)
```

```
cnn_model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
history = cnn_model.fit(
    X_train,
    y_train,
    epochs=10,
    batch_size=32,
    validation_split=0.2
)
```

```
test_loss, test_accuracy = cnn_model.evaluate(
    X_test,
    y_test,
    verbose=0
)
print("Test Loss:", test_loss)
print("Test Accuracy:", test_accuracy)
print("Test Accuracy (%):", test_accuracy * 100)
```

```
Test Loss: 0.1134316697716713
Test Accuracy: 0.9729999899864197
Test Accuracy (%): 97.29999899864197
```

```
from sklearn.metrics import confusion_matrix, classification_report
import numpy as np
y_pred_probs = cnn_model.predict(X_test)
y_pred = np.argmax(y_pred_probs, axis=1)
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

```
63/63 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step
Confusion Matrix:
[[174   0   0   0   0   0   1   0   0   0]
 [  0 232   1   0   0   1   0   0   0   0]
 [  0   1 216   0   0   0   0   2   0   0]
 [  0   0   0 202   0   3   0   2   0   0]
 [  0   0   0   0 213   0   3   0   0   1]
 [  1   0   1   1   0 174   1   1   0   0]
 [  2   1   0   0   0   2 173   0   0   0]
 [  0   1   2   2   0   0   0 200   0   0]
 [  4   1   5   1   1   0   0   2 176   2]
 [  1   0   0   0   1   3   0   2   1 186]]
```

```
plt.figure(figsize=(12, 6))
for i in range(15):
    plt.subplot(3, 5, i + 1)
    plt.imshow(X_test[i].reshape(28, 28), cmap="gray")
    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {y_pred[i]}"
    )
    plt.axis("off")
plt.tight_layout()
plt.show()
```

